



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

**Faculty of
Computing**

SECP3623

Database Programming

2025/2026 SEMESTER 1

Assignment 1 (10%)

Name	:	LEE YIN SHEN
Matric Id	:	A23CS0236
Section	:	1

Name	:	-
Matric Id	:	-
Section	:	-

MARKING RUBRICS

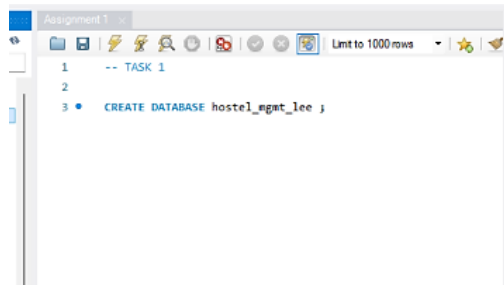
Task	Criteria	Marks	Total Marks
Task 1 Database Creation & Constraints	Creation of database and all tables with valid keys, datatypes, and constraints (PK, FK, NOT NULL, UNIQUE). Logical order and referential integrity preserved.	3	
	ALTER TABLE (add email UNIQUE) and DROP TABLE demonstration executed successfully and documented.	2	
	Insertion order, valid FK references, and no orphan data.	2	
Task 2 Data Manipulation & Filtering	Five tables populated with ≥10 valid records each; varied and meaningful data UPDATE/DELETE executed correctly with logical results.	1	
	Accurate use of WHERE, BETWEEN, LIKE, IN, AND/OR/NOT queries.	3	
	Correct use of aggregate and string functions (COUNT, CONCAT, and more) .	2	
Task 3 Reporting & Aggregation	VIEW creation	2	
	Use of GROUP BY, HAVING, ROUND, and CASE with accurate aggregated results.	3	
	Clear labelling, readable outputs, screenshots show executed results.	2	
Total (20)			

Task 1: Database Creation and Integrity Constraints (CLO1)

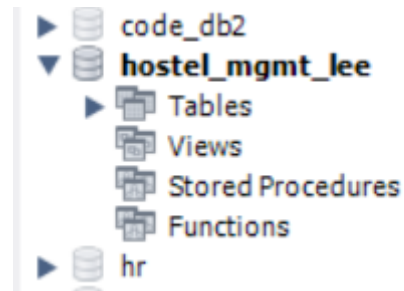
SQL command

Output / Result

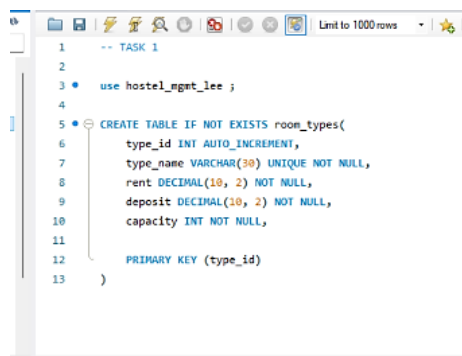
Create database 'hostel_mgmt_lee' using CREATE DATABASE



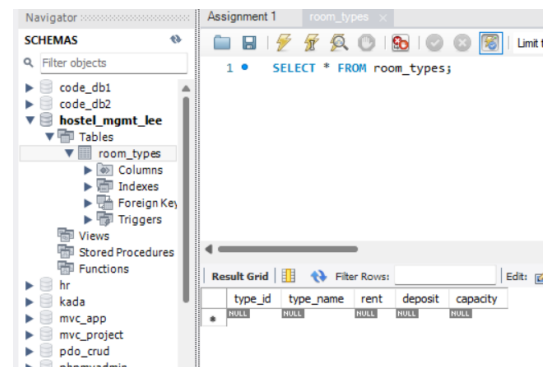
```
-- TASK 1
1
2
3 • CREATE DATABASE hostel_mgmt_lee ;
```



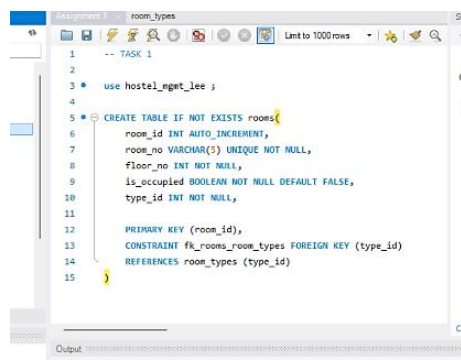
Create table 'room_types' using CREATE TABLE, then set primary key as type_id.



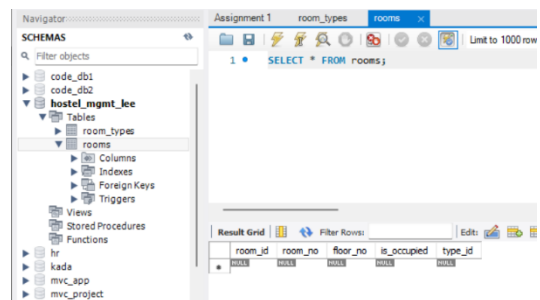
```
-- TASK 1
1
2
3 • use hostel_mgmt_lee ;
4
5 • CREATE TABLE IF NOT EXISTS room_types(
6     type_id INT AUTO_INCREMENT,
7     type_name VARCHAR(30) UNIQUE NOT NULL,
8     rent DECIMAL(10, 2) NOT NULL,
9     deposit DECIMAL(10, 2) NOT NULL,
10    capacity INT NOT NULL,
11
12    PRIMARY KEY (type_id)
13 )
```



Create table 'rooms' using CREATE TABLE, then set primary key as room_id, type_id as foreign key, reference to room_types table.



```
-- TASK 1
1
2
3 • use hostel_mgmt_lee ;
4
5 • CREATE TABLE IF NOT EXISTS rooms(
6     room_id INT AUTO_INCREMENT,
7     room_no VARCHAR(5) UNIQUE NOT NULL,
8     floor_no INT NOT NULL,
9     is_occupied BOOLEAN NOT NULL DEFAULT FALSE,
10    type_id INT NOT NULL,
11
12    PRIMARY KEY (room_id),
13    CONSTRAINT fk_rooms_room_types FOREIGN KEY (type_id)
14    REFERENCES room_types (type_id)
15 )
```



Create table 'students' using CREATE TABLE, then set primary key as student_id, foreign key as room_id, reference to rooms table.

Create table 'maintenance' using CREATE TABLE, then set primary key as maint_id, foreign key as room_id, reference to rooms table.

Create table 'payments' using CREATE TABLE, set primary key as payment_id, foreign key as student_id, reference to students table.

Add 'email' column into students table using ALTER TABLE, then set as Unique.

Create a temporary testing table using CREATE TABLE, then drop it using DROP TABLE.

```
Assignment 1* x room_types rooms students maintenance
1 -- TASK 1
2
3 • use hostel_mgmt_lee ;
4
5 • CREATE TABLE temp_test_table(
6     temp_id INT PRIMARY KEY
7 )
8
9
```

```
Assignment 1 x rooma students maintenance
1 -- TASK 1
2
3 • use hostel_mgmt_lee ;
4
5 • DROP TABLE temp_test_table ;
6
7
```

Navigation pane showing the database schema structure:

- code_db1
 - code_db2
 - hostel_mgmt_lee
 - Tables
 - maintenance
 - payments
 - room_types
 - rooms
 - students
 - temp_test_table
 - Views
 - Stored Procedures
 - Functions

Query window showing the result of a query:

```
1 • SELECT * FROM temp_test_table;
```

Result grid showing the output:

temp_id
1

Task 2: Data Manipulation and Filtering (CLO2)

SQL command

Output / Result

Insert data into room_types table using INSERT INTO ... VALUES

```
-- TASK 2
1
2
3 use hostel_mngmt_lee ;
4
5 INSERT INTO room_types (type_name, rent, deposit, capacity) VALUES
6 ('Single A', 450.00, 250.00, 1),
7 ('Single B', 550.00, 300.00, 1),
8 ('Single C', 650.00, 500.00, 1),
9 ('Double A', 700.00, 400.00, 2),
10 ('Double B', 850.00, 500.00, 2),
11 ('Double C', 600.00, 350.00, 2),
12 ('Premium A', 950.00, 500.00, 1),
13 ('Premium B', 1200.00, 600.00, 2),
14 ('Family A', 1600.00, 800.00, 4),
15 ('Family B', 2000.00, 1300.00, 3);
16
```

Assignment 1 room_types rooms students

1 SELECT * FROM room_types;

Result Grid Filter Rows: Edit:

type_id	type_name	rent	deposit	capacity
1	Single A	450.00	250.00	1
2	Single B	550.00	300.00	1
3	Single C	650.00	500.00	1
4	Double A	700.00	400.00	2
5	Double B	850.00	500.00	2
6	Double C	600.00	350.00	2
7	Premium A	950.00	500.00	1
8	Premium B	1200.00	600.00	2
9	Family A	1600.00	800.00	4
10	Family B	2000.00	1300.00	3

Insert data into rooms table using INSERT INTO ... VALUES

```
Assignment 1 room_types rooms students maintenance payments
1 -- TASK 2
2
3 use hostel_mngmt_lee ;
4
5 INSERT INTO rooms (room_no, floor_no, type_id) VALUES
6 ('A101', 1, 1),
7 ('A102', 1, 1),
8 ('A103', 1, 1),
9 ('B201', 2, 3),
10 ('B202', 2, 3),
11 ('B203', 2, 3),
12 ('C301', 3, 5),
13 ('C302', 3, 5),
14 ('D401', 4, 7),
15 ('D402', 4, 7);
16
17
```

Assignment 1 room_types rooms students

1 SELECT * FROM rooms;

Result Grid Filter Rows: Edit:

room_id	room_no	floor_no	is_occupied	type_id
1	A101	1	0	1
2	A102	1	0	1
3	A103	1	0	1
4	B201	2	0	3
5	B202	2	0	3
6	B203	2	0	3
7	C301	3	0	5
8	C302	3	0	5
9	D401	4	0	7
10	D402	4	0	7

Insert data into students table using INSERT INTO ... VALUES

```
Assignment 1 room_types rooms students maintenance payments temp_test_table
1 -- TASK 2
2
3 use hostel_mngmt_lee ;
4
5 INSERT INTO students (fname, lname, status, checkin_date, room_id, email) VALUES
6 ('Ahmad', 'Zaki', 'ACTIVE', '2025-10-01', 1, 'ahmad.zaki@graduate.utm.my'),
7 ('Siti', 'Nurhaliza', 'ACTIVE', '2025-10-01', 2, 'siti.nurhaliza@graduate.utm.my'),
8 ('Amin', 'Yusof', 'ACTIVE', '2025-10-02', 4, 'amin.yusof@graduate.utm.my'),
9 ('Tan', 'Wei Ling', 'ACTIVE', '2025-10-02', 7, 'tan.wei ling@graduate.utm.my'),
10 ('Kumar', 'Rao', 'ACTIVE', '2025-10-03', 7, 'kumar.rao@graduate.utm.my'),
11 ('David', 'Lim', 'ACTIVE', '2025-10-05', 8, 'david.lim@graduate.utm.my'),
12 ('Chong', 'Ee Von', 'ACTIVE', '2025-10-05', 8, 'chong.ee von@graduate.utm.my'),
13 ('Alice', 'Gomez', 'ACTIVE', '2025-10-06', 9, 'alice.gomez@graduate.utm.my'),
14 ('Farah', 'Lee', 'ACTIVE', '2025-10-08', 10, 'farah.lee@graduate.utm.my'),
15 ('Zainab', 'Ali', 'NON_ACTIVE', '2024-09-15', 3, 'zainab.ali@graduate.utm.my'),
16 ('Michael', 'Tan', 'NON_ACTIVE', '2024-08-20', NULL, 'michael.tan@graduate.utm.my');
17
18
19
20
```

Assignment 1 room_types rooms students maintenance payments temp_test_table

1 SELECT * FROM students;

Result Grid Filter Rows: Edit: Export/Import: Wrap Cell Co

student_id	fname	lname	status	checkin_date	room_id	email
1	Ahmad	Zaki	ACTIVE	2025-10-01	1	ahmad.zaki@graduate.utm.my
2	Siti	Nurhaliza	ACTIVE	2025-10-01	2	siti.nurhaliza@graduate.utm.my
3	Amin	Yusof	ACTIVE	2025-10-02	4	amin.yusof@graduate.utm.my
4	Tan	Wei Ling	ACTIVE	2025-10-02	7	tan.wei ling@graduate.utm.my
5	Kumar	Rao	ACTIVE	2025-10-03	7	kumar.rao@graduate.utm.my
6	David	Lim	ACTIVE	2025-10-05	8	david.lim@graduate.utm.my
7	Chong	Ee Von	ACTIVE	2025-10-05	8	chong.ee von@graduate.utm.my
8	Alice	Gomez	ACTIVE	2025-10-06	9	alice.gomez@graduate.utm.my
9	Farah	Lee	ACTIVE	2025-10-08	10	farah.lee@graduate.utm.my
10	Zainab	Ali	NON_ACTIVE	2024-09-15	3	zainab.ali@graduate.utm.my
11	Michael	Tan	NON_ACTIVE	2024-08-20	NULL	michael.tan@graduate.utm.my

Insert data into maintenance table using INSERT INTO ... VALUES

```

1  -- TASK 2
2
3  * use hostel_mgmt_lee ;
4
5  * INSERT INTO maintenance (room_id, issue_desc, severity, status, reported_on, resolved_on)
6  VALUES
7  (1, 'Water leakage in bathroom', 'MEDIUM', 'OPEN', '2025-10-10', NULL),
8  (2, 'Window latch broken', 'MEDIUM', 'RESOLVED', '2025-10-01', '2025-10-03'),
9  (4, 'Air-conditioner not cold', 'HIGH', 'OPEN', '2025-10-11', NULL),
10 (7, 'Study chair broken', 'LOW', 'RESOLVED', '2025-10-05', '2025-10-06'),
11 (8, 'Wi-Fi slow', 'LOW', 'OPEN', '2025-10-12', NULL),
12 (9, 'Shower head leaking', 'LOW', 'OPEN', '2025-10-13', NULL),
13 (6, 'Key card not working', 'HIGH', 'RESOLVED', '2025-10-07', '2025-10-07'),
14 (2, 'Bunk bed ladder loose', 'MEDIUM', 'OPEN', '2025-10-14', NULL),
15 (1, 'Sink clogged', 'LOW', 'OPEN', '2025-10-15', NULL),
16 (5, 'Wardrobe door hinge broken', 'MEDIUM', 'RESOLVED', '2025-08-01', '2025-08-03');
17
18
19

```

Assignment 1 room_types rooms students maintenance payments temp_test_table

1 * SELECT * FROM maintenance;

maint_id	issue_desc	severity	status	reported_on	resolved_on	room_id
1	Water leakage in bathroom	MEDIUM	OPEN	2025-10-10	NULL	1
2	Window latch broken	MEDIUM	RESOLVED	2025-10-01	2025-10-03	2
3	Air-conditioner not cold	HIGH	OPEN	2025-10-11	NULL	4
4	Study chair broken	LOW	RESOLVED	2025-10-05	2025-10-06	7
5	Wi-Fi slow	LOW	OPEN	2025-10-12	NULL	8
6	Shower head leaking	LOW	OPEN	2025-10-13	NULL	9
7	Key card not working	HIGH	RESOLVED	2025-10-07	2025-10-07	6
8	Bunk bed ladder loose	MEDIUM	OPEN	2025-10-14	NULL	2
9	Sink clogged	LOW	OPEN	2025-10-15	NULL	1
10	Wardrobe door hinge broken	MEDIUM	RESOLVED	2025-08-01	2025-08-03	5

Insert data into payments table using INSERT INTO ... VALUES

```

1  -- Task 1
2
3
4  * USE hostel_mgmt_lee ;
5
6  * INSERT INTO payments (amount, paid_on, method, note, student_id) VALUES
7  (700.00, '2025-10-01', 'FPX', 'Deposit + Oct Rent', 1),
8  (700.00, '2025-10-01', 'CARD', 'Deposit + Oct Rent', 2),
9  (1150.00, '2025-10-02', 'FPX', 'Deposit + Oct Rent', 3),
10 (1250.00, '2025-10-02', 'TNG', 'Deposit + Oct Rent', 4),
11 (1350.00, '2025-10-03', 'CARD', 'Deposit + Oct Rent', 5),
12 (1350.00, '2025-10-05', 'FPX', 'Deposit + Oct Rent', 6),
13 (1350.00, '2025-10-05', 'CASH', 'Deposit + Oct Rent', 7),
14 (1450.00, '2025-10-06', 'FPX', 'Deposit + Oct Rent', 8),
15 (450.00, '2025-11-01', 'FPX', 'Nov Rent', 1),
16 (450.00, '2025-11-01', 'CARD', 'Nov Rent', 2);
17

```

1 * SELECT * FROM payments;

payment_id	amount	paid_on	method	note	student_id
1	700.00	2025-10-01	FPX	Deposit + Oct Rent	1
2	700.00	2025-10-01	CARD	Deposit + Oct Rent	2
3	1150.00	2025-10-02	FPX	Deposit + Oct Rent	3
4	1250.00	2025-10-02	TNG	Deposit + Oct Rent	4
5	1350.00	2025-10-03	CARD	Deposit + Oct Rent	5
6	1350.00	2025-10-05	FPX	Deposit + Oct Rent	6
7	1350.00	2025-10-05	CASH	Deposit + Oct Rent	7
8	1450.00	2025-10-06	FPX	Deposit + Oct Rent	8
9	450.00	2025-11-01	FPX	Nov Rent	1
10	450.00	2025-11-01	CARD	Nov Rent	2

UPDATE 'is_occupied' to TRUE for rooms with active students using SET, if not set to FALSE. The condition filtering is done by using SELECT, FROM, WHERE.

```

4
5  * UPDATE rooms
6  SET is_occupied = TRUE
7  WHERE room_id IN(
8  SELECT DISTINCT room_id
9  FROM students
10 WHERE status = 'ACTIVE' AND room_id IS NOT NULL
11 );
12
13 * UPDATE rooms
14 SET is_occupied = FALSE
15 WHERE room_id NOT IN(
16 SELECT DISTINCT room_id
17 FROM students
18 WHERE status = 'ACTIVE' AND room_id IS NOT NULL
19 );
20

```

1 * SELECT * FROM rooms;

room_id	room_no	floor_no	is_occupied	type_id
1	A101	1	1	1
2	A102	1	1	1
3	A103	1	0	1
4	B201	2	1	3
5	B202	2	0	3
6	B203	2	0	3
7	C301	3	1	5
8	C302	3	1	5
9	D401	4	1	7
10	D402	4	1	7

Delete old maintenance records that older than 60 days using DELETE FROM. The condition filtering is done by WHERE ... AND. I use < DATE_SUB(NOW(), INTERVAL 60 DAY) to identify whether the reported already resolved over 60 days or not.


```

1  -- TASK 2
2
3  • use hostel_mgmt_lee ;
4
5  • DELETE FROM maintenance
6    WHERE status = 'RESOLVED'
7    AND reported_on < DATE_SUB(NOW(), INTERVAL 60 DAY) ;
8
9
10

```

```

1  • SELECT * FROM maintenance;

```


maint_id	issue_desc	severity	status	reported_on	resolved_on	room_id
1	Water leakage in bathroom	MEDIUM	OPEN	2025-10-10	N/A	1
2	Window latch broken	MEDIUM	RESOLVED	2025-10-01	2025-10-03	2
3	Air-conditioner not cold	HIGH	OPEN	2025-10-11	N/A	4
4	Study chair broken	LOW	RESOLVED	2025-10-05	2025-10-06	7
5	Wi-Fi slow	LOW	OPEN	2025-10-12	N/A	8
6	Shower head leaking	LOW	OPEN	2025-10-13	N/A	9
7	Key card not working	HIGH	RESOLVED	2025-10-07	2025-10-07	6
8	Bunk bed ladder loose	MEDIUM	OPEN	2025-10-14	N/A	2
9	Sink clogged	LOW	OPEN	2025-10-15	N/A	1
*	N/A	N/A	N/A	N/A	N/A	N/A

Filtering Query using SELECT, FROM, WHERE and BETWEEN

```

1  -- TASK 2
2
3  • use hostel_mgmt_lee ;
4
5  • SELECT type_name, rent
6    FROM room_types
7    WHERE rent BETWEEN 400.00 AND 600.00 ;
8

```


type_name	rent
Single A	450.00
Single B	550.00
Single C	650.00
Double A	700.00
Double C	600.00

type_name	rent
Single A	450.00
Single B	550.00
Single C	650.00
Double A	700.00
Double C	600.00

Filtering Query using SELECT, FROM, WHERE and LIKE

```

1  -- TASK 2
2
3  • use hostel_mgmt_lee ;
4
5  • SELECT fname, lname, status
6    FROM students
7    WHERE fname LIKE 'AS';
8

```


fname	lname	status
Ahmad	Zaki	ACTIVE
Amin	Yusof	ACTIVE
Alice	Gomez	ACTIVE

fname	lname	status
Ahmad	Zaki	ACTIVE
Amin	Yusof	ACTIVE
Alice	Gomez	ACTIVE

Filtering Query using SELECT, FROM, WHERE and IN

```

1  -- TASK 2
2
3  • use hostel_mgmt_lee ;
4
5  • SELECT student_id, amount, paid_on, method
6    FROM payments
7    WHERE method IN ('FPX', 'CARD') ;
8
9
10

```


student_id	amount	paid_on	method
1	700.00	2025-10-01	FPX
2	700.00	2025-10-01	CARD
3	1150.00	2025-10-02	FPX
5	1350.00	2025-10-03	CARD
6	1350.00	2025-10-05	FPX
8	1450.00	2025-10-06	FPX
1	450.00	2025-11-01	FPX
2	450.00	2025-11-01	CARD

student_id	amount	paid_on	method
1	700.00	2025-10-01	FPX
2	700.00	2025-10-01	CARD
3	1150.00	2025-10-02	FPX
5	1350.00	2025-10-03	CARD
6	1350.00	2025-10-05	FPX
8	1450.00	2025-10-06	FPX
1	450.00	2025-11-01	FPX
2	450.00	2025-11-01	CARD

Filtering Query using SELECT, FROM, WHERE and combination of AND and OR

Assignment 1 - room_types rooms students maintenance payments temp...

```

1 -- TASK 2
2
3 use hostel_mgmt_lee;
4
5 SELECT room_id, issue_desc, severity, status
6 FROM maintenance
7 WHERE status = 'OPEN'
8 AND (severity = 'HIGH' OR severity = 'MEDIUM');
9

```

Result Grid | Filter Rows: | Export: | Wrap Cell Contents: ☐

room_id	issue_desc	severity	status
1	Water leakage in bathroom	MEDIUM	OPEN
4	Air-conditioner not cold	HIGH	OPEN
2	Bunk bed ladder loose	MEDIUM	OPEN

Result Grid | Filter Rows: | Export: |

room_id	issue_desc	severity	status
1	Water leakage in bathroom	MEDIUM	OPEN
4	Air-conditioner not cold	HIGH	OPEN
2	Bunk bed ladder loose	MEDIUM	OPEN

Use aggregate function COUNT(*) to calculate total active students

Assignment 1 - room_types rooms students maintenance payments

```

1 -- TASK 2
2
3 use hostel_mgmt_lee;
4
5 SELECT COUNT(*) AS total_active_students
6 FROM students
7 WHERE status = 'ACTIVE';
8
9

```

Result Grid | Filter Rows: | Export: | Wrap Cell Contents: ☐

total_active_students
9

Result Grid | Filter Rows: |

total_active_students
9

Use string function CONCAT to combine student's name

Assignment 1 - room_types rooms students maintenance payments

```

1 -- TASK 2
2
3 use hostel_mgmt_lee;
4
5 SELECT
6 CONCAT(fname, ' ', lname) AS full_name,
7 status
8 FROM students;

```

Result Grid | Filter Rows: | Export: | Wrap Cell Contents: ☐

full_name	status
Ahmad Zaki	ACTIVE
Siti Nurhaliza	ACTIVE
Amin Yusof	ACTIVE
Tan Wei Ling	ACTIVE
Kumar Rao	ACTIVE
David Lim	ACTIVE
Chong Ee Von	ACTIVE
Alice Gomez	ACTIVE
Farah Lee	ACTIVE
Zainab Ali	NON_ACTIVE
Michael Tan	NON_ACTIVE

Result Grid | Filter Rows: |

full_name	status
Ahmad Zaki	ACTIVE
Siti Nurhaliza	ACTIVE
Amin Yusof	ACTIVE
Tan Wei Ling	ACTIVE
Kumar Rao	ACTIVE
David Lim	ACTIVE
Chong Ee Von	ACTIVE
Alice Gomez	ACTIVE
Farah Lee	ACTIVE
Zainab Ali	NON_ACTIVE
Michael Tan	NON_ACTIVE

Use string function UPPER to show uppercase room type

Assignment 1 - room_types rooms students maintenance payments temp...

```

1 -- TASK 2
2
3 use hostel_mgmt_lee;
4
5 SELECT
6 UPPER(type_name) AS room_type_uppercase,
7 rent
8 FROM room_types;

```

Result Grid | Filter Rows: | Export: | Wrap Cell Contents: ☐

room_type_uppercase	rent
SINGLE A	450.00
SINGLE B	550.00
SINGLE C	650.00
DOUBLE A	700.00
DOUBLE B	850.00
DOUBLE C	600.00
PREMIUM A	950.00
PREMIUM B	1200.00
FAMILY A	1600.00
FAMILY B	2000.00

Result Grid | Filter Rows: |

room_type_uppercase	rent
SINGLE A	450.00
SINGLE B	550.00
SINGLE C	650.00
DOUBLE A	700.00
DOUBLE B	850.00
DOUBLE C	600.00
PREMIUM A	950.00
PREMIUM B	1200.00
FAMILY A	1600.00
FAMILY B	2000.00

Task 3: Reporting and Aggregation (CLO2)

SQL command

Output / Result

Create view 'v_room_status' using CREATE VIEW ... AS, then use LEFT JOIN to join three tables which include room_types, students and maintenances to make sure all rooms included, even if they have no students and no issues. Then filter the joins to only count students who are 'ACTIVE' and issues that are 'OPEN'.

```

4 CREATE VIEW v_room_status AS
5 SELECT
6     r.room_no,
7     rt.type_name,
8     r.rent,
9     r.floor_no,
10    rt.capacity,
11    COUNT(DISTINCT s.student_id) AS n_occupants,
12    COUNT(DISTINCT m.maint_id) AS pending_issues,
13    CASE
14        WHEN COUNT(DISTINCT s.student_id) = 0 THEN TRUE
15        ELSE FALSE
16    END AS is_vacant
17 FROM rooms r
18
19 LEFT JOIN
20 room_types rt ON r.type_id = rt.type_id
21
22 LEFT JOIN
23 students s ON r.room_id = s.room_id AND s.status = 'ACTIVE'
24
25 LEFT JOIN
26 maintenance m ON r.room_id = m.room_id AND m.status = 'OPEN'
27 GROUP BY
28     r.room_no, rt.type_name, r.rent, r.floor_no, rt.capacity ;

```

Assignment 1* room_types rooms students maintenance payments temp_test_table v_room_status

1 SELECT * FROM hostel_mgt_lee.v_room_status;

Result Grid Filter Rows: Export: | Wrap Cell Contents: |

room_no	type_name	rent	floor_no	capacity	n_occupants	pending_issues	is_vacant
A101	Single A	450.00	1	1	1	2	0
A102	Single A	450.00	1	1	1	1	0
A103	Single A	450.00	1	1	0	0	1
B201	Single C	650.00	2	1	1	1	0
B202	Single C	650.00	2	1	0	0	1
B203	Single C	650.00	2	1	0	0	1
C301	Double B	850.00	3	2	2	0	0
C302	Double B	850.00	3	2	2	1	0
D401	Premium A	950.00	4	1	1	1	0
D402	Premium A	950.00	4	1	1	0	0

Use INNER JOIN to joins the room_types, rooms, and students tables together. Then filter to find only the 'ACTIVE' students, then GROUP BY room type to COUNT how many active students live in each room type.

```

3 use hostel_mgt_lee ;
4
5 SELECT
6     rt.type_name,
7     COUNT(s.student_id) AS total_active_students
8 FROM
9     room_types rt
10 INNER JOIN
11     rooms r ON rt.type_id = r.type_id
12 INNER JOIN
13     students s ON r.room_id = s.room_id
14 WHERE
15     s.status = 'ACTIVE'
16 GROUP BY
17     rt.type_name

```

Result Grid Filter Rows:

	type_name	total_active_students
▶	Single A	2
	Single C	1
	Double B	4
	Premium A	2

Use ROUND, AVG, and SUM on the room_types table. Then GROUP BY type_name to calculate the average rent and total deposit for each room type.

```

1 -- TASK 3
2
3 use hostel_mgt_lee ;
4
5 SELECT
6     type_name,
7     ROUND(AVG(rent), 2) AS avr_rent,
8     ROUND(SUM(deposit), 2) AS total_deposit
9 FROM
10    room_types
11 GROUP BY
12    type_name ;

```

Result Grid Filter Rows: Export: | Wrap Cell Contents: |

type_name	avr_rent	total_deposit
▶ Double A	700.00	400.00
Double B	850.00	500.00
Double C	600.00	350.00
Family A	1600.00	800.00
Family B	2000.00	1300.00
Premium A	950.00	500.00
Premium B	1200.00	600.00
Single A	450.00	250.00
Single B	550.00	300.00
Single C	650.00	500.00

Result Grid Filter Rows:

	type_name	avr_rent	total_deposit
▶	Double A	700.00	400.00
	Double B	850.00	500.00
	Double C	600.00	350.00
	Family A	1600.00	800.00
	Family B	2000.00	1300.00
	Premium A	950.00	500.00
	Premium B	1200.00	600.00
	Single A	450.00	250.00
	Single B	550.00	300.00
	Single C	650.00	500.00

Use YEAR, MONTH, and SUM on the payments table. Then GROUP BY both year and month to find the total payment for each period.

Assignment 1 | room_types | rooms | students | maintenance | payments

```

2 use hostel_mgmt_lee ;
3
4
5 SELECT
6     YEAR(paid_on) AS payment_year,
7     MONTH(paid_on) AS payment_month,
8     SUM(amount) AS total_payments
9 FROM
10    payments
11 GROUP BY
12     YEAR(paid_on), MONTH(paid_on)
13

```

Result Grid | Filter Rows: | Export | Wrap Cell Contents

	payment_year	payment_month	total_payments
▶	2025	10	9300.00
	2025	11	900.00

Result Grid | Filter Rows: | Export

	payment_year	payment_month	total_payments
▶	2025	10	9300.00
	2025	11	900.00

Use INNER JOIN to join rooms and maintenance tables. Then use WHERE to filter for 'OPEN' issues, then GROUP BY floor_no to COUNT the issues on each floor. Then use HAVING to show only floors with more than 2 issues.

Assignment 1 | room_types | rooms | students | maintenance | payments

```

2 use hostel_mgmt_lee ;
3
4
5 SELECT
6     r.floor_no,
7     COUNT(m.maint_id) AS oi_count
8 FROM
9     rooms r
10 INNER JOIN
11     maintenance m ON r.room_id = m.room_id
12 WHERE
13     m.status = 'OPEN'
14 GROUP BY
15     r.floor_no
16 HAVING
17

```

Result Grid | Filter Rows: | Export | Wrap Cell Contents

	floor_no	oi_count
▶	1	3

Result Grid | Filter Rows: | Export

	floor_no	oi_count
▶	1	3

Use ROUND, AVG and SUM function to show average rent and total deposit for each room type.

Assignment 1 | room_types | rooms | students | maintenance | payments

```

2 use hostel_mgmt_lee ;
3
4
5 SELECT
6     type_name,
7     ROUND(AVG(rent), 2) AS avr_rent,
8     ROUND(SUM(deposit), 2) AS total_deposit
9 FROM
10    room_types
11 GROUP BY
12     type_name ;
13

```

Result Grid | Filter Rows: | Export | Wrap Cell Contents

	type_name	avr_rent	total_deposit
▶	Double A	700.00	400.00
	Double B	850.00	500.00
	Double C	600.00	350.00
	Family A	1600.00	800.00
	Family B	2000.00	1300.00
	Premium A	950.00	500.00
	Premium B	1200.00	600.00
	Single A	450.00	250.00
	Single B	550.00	300.00
	Single C	650.00	500.00

Result Grid | Filter Rows: | Export

	type_name	avr_rent	total_deposit
▶	Double A	700.00	400.00
	Double B	850.00	500.00
	Double C	600.00	350.00
	Family A	1600.00	800.00
	Family B	2000.00	1300.00
	Premium A	950.00	500.00
	Premium B	1200.00	600.00
	Single A	450.00	250.00
	Single B	550.00	300.00
	Single C	650.00	500.00

Use UPPER to show uppercase full name.

Assignment 1 x room_types rooms students maintenance payments

Limit to 1000 rows

```

1 -- TASK 3
2
3 • use hostel_mgmt_lee ;
4
5 • SELECT
6     UPPER(fname) AS firs_upper,
7     UPPER(lname) AS last_upper
8 FROM
9     students
10 WHERE
11     status = 'ACTIVE' ;

```

Result Grid | Filter Rows: | Export: | Wrap Cell Contents:

firs_upper	last_upper
AHMAD	ZAKI
SITI	NURHALIZA
AMIN	YUSOF
TAN	WEI LING
KUMAR	RAO
DAVID	LIM
CHONG	EE VON
ALICE	GOMEZ
FARAH	LEE

Result Grid | Filter Rows:

firs_upper	last_upper
AHMAD	ZAKI
SITI	NURHALIZA
AMIN	YUSOF
TAN	WEI LING
KUMAR	RAO
DAVID	LIM
CHONG	EE VON
ALICE	GOMEZ
FARAH	LEE

Use CONCAT to combine first and last name to show full name.

Assignment 1 x room_types rooms students maintenance payments

Limit to 1000 rows

```

1 -- TASK 3
2
3 • use hostel_mgmt_lee ;
4
5 • SELECT
6     CONCAT(fname, ' ', lname) AS full_name,
7     email
8 FROM
9     students ;

```

Result Grid | Filter Rows: | Export: | Wrap Cell Contents:

full_name	email
Ahmad Zaki	ahmad.zaki@graduate.utm.my
Siti Nurhaliza	siti.nurhaliza@graduate.utm.my
Amin Yusof	amin.yusof@graduate.utm.my
Tan Wei Ling	tan.wei ling@graduate.utm.my
Kumar Rao	kumar.rao@graduate.utm.my
David Lim	david.lim@graduate.utm.my
Chong Ee Von	chong.eevon@graduate.utm.my
Alice Gomez	alice.gomez@graduate.utm.my
Farah Lee	farah.lee@graduate.utm.my
Zainab Ali	zainab.ali@graduate.utm.my
Michael Tan	michael.tan@graduate.utm.my

Result Grid | Filter Rows:

full_name	email
Ahmad Zaki	ahmad.zaki@graduate.utm.my
Siti Nurhaliza	siti.nurhaliza@graduate.utm.my
Amin Yusof	amin.yusof@graduate.utm.my
Tan Wei Ling	tan.wei ling@graduate.utm.my
Kumar Rao	kumar.rao@graduate.utm.my
David Lim	david.lim@graduate.utm.my
Chong Ee Von	chong.eevon@graduate.utm.my
Alice Gomez	alice.gomez@graduate.utm.my
Farah Lee	farah.lee@graduate.utm.my
Zainab Ali	zainab.ali@graduate.utm.my
Michael Tan	michael.tan@graduate.utm.my

Use CASE with conditions to create new column and assign the column with new label.

Assignment 1 x room_types rooms students maintenance payments

Limit to 1000 rows

```

5 • SELECT
6     type_name,
7     rent,
8     CASE
9         WHEN rent < 600.00 THEN 'LOW'
10        WHEN rent < 1000.00 THEN 'MEDIUM'
11        ELSE 'HIGH'
12    END AS rent_category
13 FROM
14     room_types
15 ORDER BY
16     rent ;

```

Result Grid | Filter Rows: | Export: | Wrap Cell Contents:

type_name	rent	rent_category
Single A	450.00	LOW
Single B	550.00	LOW
Double C	600.00	MEDIUM
Single C	650.00	MEDIUM
Double A	700.00	MEDIUM
Double B	850.00	MEDIUM
Premium A	950.00	MEDIUM
Premium B	1200.00	HIGH
Family A	1600.00	HIGH
Family B	2000.00	HIGH

Result Grid | Filter Rows:

type_name	rent	rent_category
Single A	450.00	LOW
Single B	550.00	LOW
Double C	600.00	MEDIUM
Single C	650.00	MEDIUM
Double A	700.00	MEDIUM
Double B	850.00	MEDIUM
Premium A	950.00	MEDIUM
Premium B	1200.00	HIGH
Family A	1600.00	HIGH
Family B	2000.00	HIGH

Table Created

Table Name	Number of records
room_types	10
rooms	10
students	11
maintenance	10
payments	10